Recommending TV Programmes on sweet.tv: Viewers Re-Watch Their Shows, and a Habit-Aware Re-Ranker Beats Personalized PageRank at MAP@5

Student: E. Shmat Team: Individual Email: shmat.evg@gmail.com Date: 2026-10-22
Assignment: Final Exam Quiz (AVITO) — TV programme recommendation, Kaggle `sweettv-tv-program-recommender` Code: github.com/ShmatEugene/hse-llm4rec-exam-sweettv

Abstract — sweet.tv must name the five TV programmes each of 1,260 users will watch most often in weeks 31–42, but its view log records only channel and time, and only 10.8% of the weeks 11–20 schedule rows carry a programme id. I joined the log to the schedule with the organisers' rule (an airing counts when at least 80% of it was seen) and compared popularity, a personal re-watch count, content-based TF-IDF, implicit ALS and Personalized PageRank on a 12-week temporal hold-out, then combined them with a gradient-boosted re-ranker scored out of fold by user. The re-ranker reaches validation MAP@5 0.38095 against 0.34310 for PPR and 0.23307 for popularity, and its returned Kaggle public score is 0.45173 (private 0.44638). Because 89.06% of its test slots are programmes the user already watched, the platform should serve a habit-aware "your shows" rail and treat discovery of new programmes as a separate problem.

Index Terms — TV recommendation, implicit feedback, personalized PageRank, learning to re-rank, MAP@K.

1. Introduction

Problem. The competition gives a view log (5,821,987 rows, 4,914 users, weeks 11–30) with `user_id, vsetv_id, start_time, stop_time, duraton` and a broadcast schedule (3,444,007 airings over weeks 11–42) [1]. An item exists only after a join: same channel, overlapping intervals. The hidden answer for weeks 31–42 is, per the competition page, each user's five programmes with the most airings watched at screen time ≥ 0.8 [1]; the score is MAP@5 [2]. The exam asks for at least two methods against popularity on one hold-out, and for a platform strategy [3]. **Motivation.** The task is not a cold catalogue problem: 1,256 of the 1,260 test users have at least one watched airing in history, and 48.95% of validation target slots are programmes the user already watched. Whether a model should repeat the familiar or find the new is therefore a measurable question, not a slogan. **Example (input → output).** View row: channel 3, 2020-03-23 16:35:30 → 17:15:50; airing of programme 2400480 (*TCH*, news) on channel 3 at 16:45:00 for 1500 s; the overlap covers the airing, so it is one watched airing. Output for that user (validation, personal method): `2400480 2400467 2400508 200337 5900347`, of which 2400480 and 2400467 are in the user's actual weeks 19–30 top-5. **Contributions.** (i) A join reproduced by two independent implementations with 0 differences over 230,113 user–programme pairs; (ii) six rankers and a re-ranking hybrid on one 4,254-user hold-out with bootstrap intervals and seven returned Kaggle scores; (iii) a strategy argument resting on offline reach, coverage and repeat-share numbers, not on an online claim.

2. Related Work

Hu, Koren and Volinsky [4] treat implicit feedback as a binary preference p_{ui} with confidence $c_{ui} = 1 + \alpha r_{ui}$ (or $1 + \alpha \log(1 + r_{ui}/\varepsilon)$) and solve it by alternating least squares; they report $\alpha = 40$ as a good value on their TV data, which I tested and which fails here (§4). The course lecture on graph recommenders [5] covers random walks and PageRank and personalises them through the restart distribution (user-seeded PPR, α = link-following probability); I use a one-hot user restart on the user–programme graph. The competition pages [1] fix the data provenance, the screen-time ≥ 0.8 rule and the strict top-5 answer; Hamner's `apk` [2] is the Kaggle reference MAP@K (division by $\min(|A|, K)$, repeated predictions not re-counted). The re-ranker uses scikit-learn's histogram gradient boosting [6]; the exam statement [3] fixes K = 5 and the protocol. **Alternatives considered.** *Association rules*: rejected as a scorer because the target is a per-user repeat count and rules state co-occurrence of different items; they would never rank a user's own daily news first. *Two-tower retrieval*: unnecessary at 7,446 items, where exact scoring of every candidate takes seconds. *Tuning the screen-time threshold*: rejected because the answer key is defined at 0.8 [1]; changing it would change what is predicted.

3. Method

Join. For a view $[s, e)$ on channel c, every airing $[a, a + d)$ on c with $a < e$ and $a + d > s$ is matched (binary search over the schedule sorted by channel and start; ends are non-decreasing within a channel, checked); overlaps of several rows of one user on one airing are summed (zapping away and back), screen time = watched / d, and an airing is *watched* at ≥ 0.8. Rows with `tv_show_id = 0` are unlabelled airtime: they are matched (so the counters close) and then dropped as items, as the data page instructs [1]; their airtime still enters the channel and hour habit features. **Validation split.** History = the log clipped at Monday of week 19 (weeks 11–18, views cut at the boundary so no later second leaks in); target = airings starting in weeks 19–30, i.e. 12 weeks, the same length as the test window. Ground truth is built exactly like the organisers': top-5 programmes by number of watched airings (ties by watched seconds, then id); users with ≥ 5 such programmes form the cohort (4,254). For the test, history is weeks 11–30. **Pool, repeats, fallback.** Candidates are labelled programmes with ≥ 1 airing in the target window (validation 7,045; test 6,642), taken from the schedule only — the weeks 31–42 schedule is used for nothing else. Repeats of watched programmes are allowed for every method. Every method ranks the pool by its score; non-positive scores are never preferred, and every list is padded from one shared fallback (popularity order, ties by id), so each user gets exactly five distinct pool ids. **Methods.** *Popularity*: distinct history users with a watched airing. *Personal*: the user's watched-airing count with an exponential half-life. *Formula* (proposed by me in the session): count × mean view rating (5 if screen time ≥ 0.99, else $2 + 9s^2$) × recency × $\log(1 + \text{pop})$. *Content*: TF-IDF over category, genres, decade, director and actor tokens; profile = count-weighted sum; cosine. *ALS* [4]: exact per-row solves with the $Y^\top Y + Y^\top (C_u - I)Y$ identity. *PPR* [5]: bipartite walk with restart at the user, iterated to an L1 change below 10^{-9}. All but popularity can be multiplied by (target-window airings)^b, $b \in \{0, 0.25, 0.5\}$ chosen on validation, because the answer counts airings: *TCH* (2400480) has 1,434 airings in weeks 19–30, a weekly film one. **Hybrid.** Candidates = programmes the user touched in history ∪ top-30 by PPR ∪ top-30 by popularity ∪ top-30 by channel exposure (target airings on the user's channels); 27 features in 8 groups (counts, view quality, recency, popularity, schedule, channel and hour habits, user activity, and the PPR/ALS/content scores); `HistGradientBoostingClassifier` (300 trees, learning rate 0.05, 31 leaves, min leaf 50, L2 1.0, seed 42) predicts "in target top-5". A second round compared, out of fold, 30 vs 60 candidates per source, 300 vs 600 trees and a Poisson regression on the target frequency; 60 candidates with the same classifier won on validation and is the final configuration. **Non-obvious decisions.** (1) The threshold is taken from the answer key, not tuned. (2) The re-ranker's labels come from weeks 19–30; on validation every user is scored by a model fitted on the other four user folds, and for the test the model is refitted on all validation pairs — weeks ≤ 30, i.e. training data from the test's point of view. Counts are divided by history length (8 vs 20 weeks) to share a scale. (3) Each method's configuration was chosen on validation; the *designated final* was then chosen among eight finished candidates by public score (Table 2), so it is selected with the public leaderboard in view and its private score is the independent check. **Environment.** Local: Apple M1, 8 GB, Python 3.9.13, NumPy 1.24.4, pandas 2.2.1, SciPy 1.9.1. Grids, re-ranker and `final.py` ran in the private Kaggle kernel `shmate/sweettv-exam-val` (4 CPUs, Python 3.13.15, NumPy 2.1.3, scikit-learn 1.6.1) because the full re-ranker run was killed locally for memory. Seeds: ALS 42, boosting 42, bootstrap 0. Runtimes (Kaggle, validation): personal 0.2 s, ALS 3.9 s, content 10.0 s, PPR 28.6 s, re-ranker 149 s.

4. Experiments

4.1 Data accounting (before any model number)

Step (full log, `results/join_output.txt`, `audit_output.txt`)	Count
Log rows → on a schedule channel (197 of 202 `vsetv_id` are `channel_id`s)	5,821,987 → 5,708,408
→ overlapping ≥ 1 airing; (view, airing) pairs; of which labelled	5,408,598; 7,752,433; 3,847,696
(user, labelled airing) pairs → with screen time ≥ 0.8 (watched)	3,014,129 → 615,979
Distinct (user, programme) pairs with ≥ 1 watched airing; users; programmes	230,113; 4,841; 3,867
Validation cohort = ≥1 watched + contact only below 0.8 + log but no labelled contact + no history log	4,254 = 4,146 + 69 + 13 + 26
Test users (template) = same four disjoint categories	1,260 = 1,256 + 2 + 2 + 0
Validation users whose 5th and 6th programme tie on frequency	2,679 of 4,254

Table 1: Join counters and cohort arithmetic. Clock check: the share of views starting < 120 s after a programme start on their channel is 0.0389 with no shift, at most 0.0358 for shifts of ±1–3 h, and 0.0212 when the channel is deliberately mismatched, so the channel key and local clocks agree. Schedule `duration` is in seconds: 99.96–100% of consecutive airings start exactly at the previous start + duration.

4.2 Results: every method, one hold-out, and every Kaggle submission

Ref	File	Method	Val MAP@5	Public	Private
57534002	01_popularity.csv	popularity	0.23307	0.30258	0.29969
57535003	02_personal.csv	personal	0.33552	0.40431	0.40044
57535268	03_ppr.csv	ppr	0.34310	0.40867	0.40662
57535949	04_rerank.csv	rerank	0.38012	0.44803	0.44354
57535951	05_formula.csv	formula	0.33170	0.39333	0.39292
57535952	06_als.csv	als	0.29758	0.31082	0.31050
57535954	07_content.csv	content	0.28242	0.34962	0.34995
57536324	08_rerank_v2.csv	rerank_v2	0.38095	0.45173	0.44638

Table 2: All eight submissions (`results/kaggle_submissions.txt`). Validation: n = 4,254, out-of-fold for the re-ranker; 95% bootstrap intervals (1,000 user resamples): popularity [0.22691, 0.23949], PPR [0.33574, 0.35007], round-1 re-ranker [0.37240, 0.38777]; round-1 re-ranker minus PPR is [+0.03347, +0.04036]. Selected configurations: personal half-life 16, b 0.25; formula half-life 4, b 0.5; content half-life 8, b 0.25; ALS 32 factors, α 1, λ 1, b 0.25; PPR α 0.3, half-life 8, b 0.25 (12 iterations, residual 6.33e-10). Validation order equals public order except ALS/content.

Designated final: `08_rerank_v2.csv` (ref 57536324, Fig. 1): best public score among the eight and also best on validation, so choosing by public did not override the validation ranking; its private score (0.44638) keeps the margin over PPR (0.40662). Round 2 itself is not a validated gain: 60 candidates raise candidate recall from 0.7474 to 0.7883 but MAP@5 only from 0.38012 to 0.38095, a difference with 95% interval [−0.00111, +0.00270]; 600 trees gave 0.37962 and the Poisson objective 0.36886. Its public step (0.44803 → 0.45173) is of the same size, so I read it as noise-level, not as evidence.

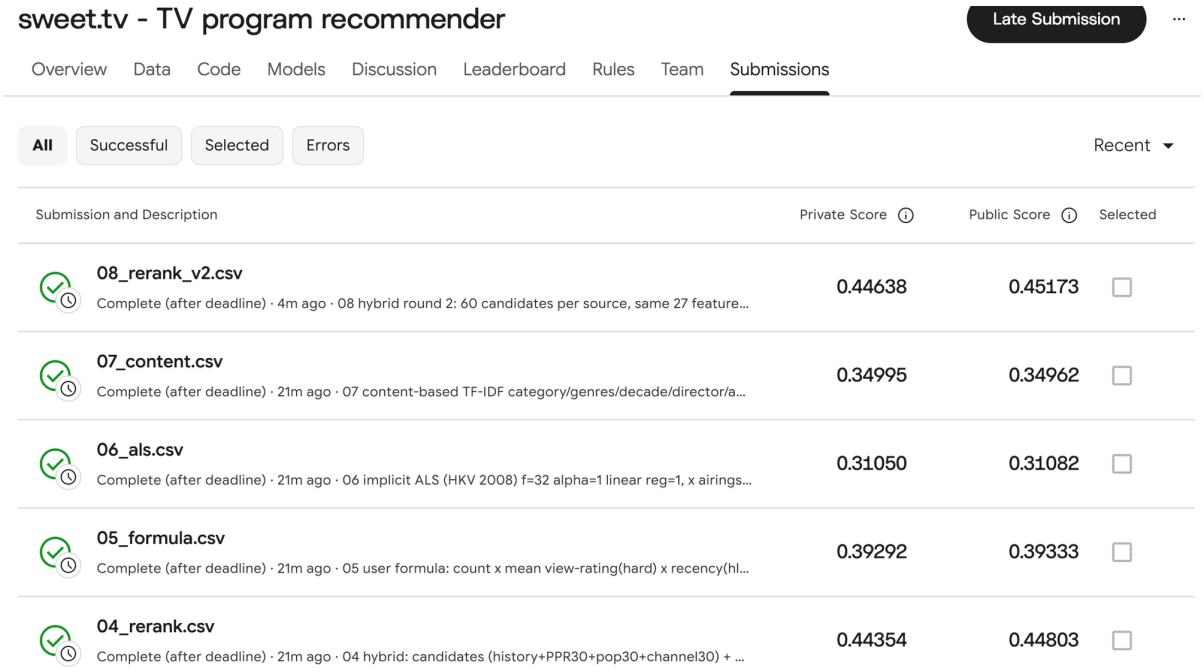


Figure 1: Returned score of the designated final as shown on Kaggle (my screenshot): `08_rerank_v2.csv`, private 0.44638, public 0.45173 — the same values as ref 57536324 in `kaggle competitions submissions` (`results/kaggle_submissions.txt`).

Every method beats popularity on public; on validation personal, formula and PPR lie between 0.33170 and 0.34310, and the re-ranker is the only method whose interval overlaps no other.

Group removed	none	counts	quality	recency	popularity	schedule	habits	user / methods
OOF MAP@5	0.38012	0.38056	0.37765	0.37945	0.38058	0.38040	0.37415	0.37922 / 0.37938

Table 3: Leave-one-group-out ablation of the re-ranker (`results/rerank_val.txt`). Channel/hour habits matter most (−0.00597); counts, popularity and schedule are redundant with other groups (removing them changes ≤ 0.0005). Candidate recall caps the hybrid: only 0.7474 of the 21,270 target slots are among its 156.3 candidates per user.

4.3 Two users, all methods (validation setting, so the actual weeks 19–30 top-5 is known)

The heavy user has 2,130 watched airings of 482 programmes in weeks 11–18 (2,130 / 56 days ≈ 38 a day, consistent with a television left on); the light user has one watched airing. Hits are starred.

Method	#1	#2	#3	#4	#5
actual 19–30	240081 Факты (Инфо)	2400480 ТСН (Инфо)	2400508 Сегодня (Инфо)	12001682 ‘Губка Боб Квадратные Штан (Детям)	20088 ‘Завтрак с 1+1’. Информаци (Развлечения)
popularity	**2400480** ТСН (Инфо)	**240081** Факты (Инфо)	2400467 ‘Викна-Новости’ (Инфо)	200344 ‘Чистonews’ (Развлечения)	**2400508** Сегодня (Инфо)
personal	**2400480** ТСН (Инфо)	**12001682** ‘Губка Боб Квадратные Штан (Детям)	**240081** Факты (Инфо)	12002955 ‘Свинка Пеппа’ (Детям)	**2400508** Сегодня (Инфо)
formula	**2400480** ТСН (Инфо)	**12001682** ‘Губка Боб Квадратные Штан (Детям)	**240081** Факты (Инфо)	12002955 ‘Свинка Пеппа’ (Детям)	**2400508** Сегодня (Инфо)
content	**2400480** ТСН (Инфо)	**240081** Факты (Инфо)	**2400508** Сегодня (Инфо)	2400467 ‘Викна-Новости’ (Инфо)	5900275 ‘Утро России’ (Инфо)
als	12002793 ‘Фиксики’ (Детям)	**12001682** ‘Губка Боб Квадратные Штан (Детям)	12001863 ‘Смешарики’ (Детям)	12002806 ‘Октонавты’, 1 сезон (Детям)	10001514 ‘Теория большого взрыва’, (Сериалы)
ppr	**2400480** ТСН (Инфо)	**12001682** ‘Губка Боб Квадратные Штан (Детям)	**240081** Факты (Инфо)	12002955 ‘Свинка Пеппа’ (Детям)	**2400508** Сегодня (Инфо)
rerank	**2400480** ТСН (Инфо)	**240081** Факты (Инфо)	**2400508** Сегодня (Инфо)	2400467 ‘Викна-Новости’ (Инфо)	**20088** ‘Завтрак с 1+1’. Информаци (Развлечения)
rerank_v2	**2400480** ТСН (Инфо)	**240081** Факты (Инфо)	**2400508** Сегодня (Инфо)	**12001682** ‘Губка Боб Квадратные Штан (Детям)	2400467 ‘Викна-Новости’ (Инфо)

Table 4: Heavy user 13866173795451203812. AP@5: popularity 0.52, personal/formula/PPR 0.76, content 0.60, ALS 0.10, re-ranker round 1 0.76, final (`rerank_v2`) 0.80.

Method	#1	#2	#3	#4	#5
actual 19–30	2400508 Сегодня (Инфо)	90080535 ‘Когда возвращается прошло (Фильмы)	90080318 ‘Всё ещё будет’ (Фильмы)	90081125 ‘Дом Надежды’, 1-3 с (Фильмы)	500346 ‘Право на власть’ (Развлечения)
popularity	2400480 ТСН (Инфо)	240081 Факты (Инфо)	2400467 ‘Викна-Новости’ (Инфо)	200344 ‘Чистonews’ (Развлечения)	**2400508** Сегодня (Инфо)
personal	**500346** ‘Право на власть’ (Развлечения)	2400480 ТСН (Инфо)	240081 Факты (Инфо)	2400467 ‘Викна-Новости’ (Инфо)	200344 ‘Чистonews’ (Развлечения)
formula	**500346** ‘Право на власть’ (Развлечения)	2400480 ТСН (Инфо)	240081 Факты (Инфо)	2400467 ‘Викна-Новости’ (Инфо)	200344 ‘Чистonews’ (Развлечения)
content	500368 Ток-шоу ‘60 минут’ с Ольго (Развлечения)	2400480 ТСН (Инфо)	240081 Факты (Инфо)	500280 ‘Пусть говорят’ (Развлечения)	700156 Пойми меня (Развлечения)
als	2400340 Служба розыска детей (Инфо)	5900347 ‘Деньги’ (Инфо)	200352 ‘Вечерний квартал’ (Развлечения)	12001682 ‘Губка Боб Квадратные Штан (Детям)	**500346** ‘Право на власть’ (Развлечения)
ppr	**500346** ‘Право на власть’ (Развлечения)	2400480 ТСН (Инфо)	240081 Факты (Инфо)	12001682 ‘Губка Боб Квадратные Штан (Детям)	20088 ‘Завтрак с 1+1’. Информаци (Развлечения)
rerank	2400480 ТСН (Инфо)	6700482 Секретные материалы (Другое)	2400467 ‘Викна-Новости’ (Инфо)	10002514 ‘Кухня’ (Сериалы)	**500346** ‘Право на власть’ (Развлечения)
rerank_v2	2400480 ТСН (Инфо)	1000589 ‘Отчаянные домохозяйки’, 1 (Сериалы)	10002514 ‘Кухня’ (Сериалы)	20088 ‘Завтрак с 1+1’. Информаци (Развлечения)	6700482 Секретные материалы (Другое)

Table 5: Light user 764273904752697078. AP@5: popularity 0.04, personal/formula/PPR 0.20, content 0.00, ALS 0.04, re-ranker round 1 0.04, final (`rerank_v2`) 0.00.

4.4 Verification

(1) *Join, second implementation* (`eval/verify_join.py`, stdlib `csv` + `bisect`, raw CSVs): 230,113 user–programme pairs in both, 0 keys in only one, 0 different frequencies, 615,979 watched airings in both. (2) *Ground truth and metric* (`eval/verify.py`): six toy cases including the all-zero template (0.0) pass; the validation target recomputed from raw CSVs matches the pipeline for all 4,254 users, including order; every method’s MAP@5 recomputed from its exported list matches. (3) *Cross-environment*: the Kaggle kernel (NumPy 2.1.3) reproduced every local validation value and wrote byte-identical popularity, personal and PPR submission files (same sha256). (4) *Hand check by me*: see §6. (5) *Numbers*: `eval/check_numbers.py` finds every number of this PDF in `results/`.

5. Discussion

Failure case 1 — a 9-pair disagreement between the joins. *Detection:* the counters agreed except (view, airing) pairs with overlap > 0: 7,752,433 (NumPy) vs 7,752,424 (stdlib). *Root cause* (`results/explain_join_diff.txt`): on channel 1020 an airing at 04:10 lasting 10,200 s ends at 07:00, after the next one starts at 06:20; nine views inside the overlap match both, and the stdlib walk starts at the latest start ≤ the view start, so it skips the longer airing. *Fix:* none needed for items — both airings are unlabelled; the NumPy join, which keeps nested airings, stays the reference. *Confirmation:* every labelled counter and all 230,113 pair frequencies agree. **Failure case 2 — a cache race on Kaggle.** *Detection:* kernel v1 logged `EOFError: No data left in file` in the content and formula grids, and the re-ranker then found no cached scores. *Root cause:* five grids started in parallel on a cold cache; one process `np.loaded` `log.npz` while another was still writing it. *Fix:* atomic writes (temporary file + `os.replace`) and one warm-up process before the parallel launch. *Confirmation:* kernel v2 exited 0 in every step and reproduced the local values (personal 0.33552, PPR 0.34310, ALS 0.29758). **Failure case 3 — the re-ranker hurts thin histories.** *Detection:* the light user drops from 0.20 (personal, PPR) to 0.04 (round 1) and 0.00 (final): the model replaces the one programme the user watched with habit-driven news. *Root cause:* its features for that user are almost all zero, so popularity-like features decide. *Proposed fix:* a switching rule — users with fewer than *k* watched airings get the PPR list. *Honest status:* a proposal, not a measurement; it is cheap to test by bucketing the 4,254 out-of-fold APs by history size. **Worked.** Taking the threshold from the answer key; the 12-week mirror of the test window; one shared fallback. **Surprised.** (i) The paper’s α = 40 for ALS gives 0.11801 here (α = 1: 0.21974): with counts up to thousands, confidence explodes for always-on users and ALS stops ordering a user’s own programmes by frequency, which is exactly what the answer measures. (ii) Recency decay did not help the personal count (no decay 0.32829 vs half-life 8 0.31632). (iii) Public scores sit above validation for every method (popularity 0.30258 vs 0.23307, re-ranker 0.44803 vs 0.38012), so the test cohort is easier than mine; the method order transferred anyway. **Next.** Candidate recall is no longer the binding limit (0.7883 bought only +0.00083); the next test is the switching rule of failure case 3 — PPR below a history threshold — measured on the out-of-fold APs.

5.1 Which strategy should the platform select, and why

Familiar vs new. Lists are mostly familiar: on the test, the share of slots the user already watched is 0.9883 for personal, 0.9425 for PPR, 0.8906 for the final re-ranker, 0.7257 for popularity; the answer itself is half familiar (48.95% on validation). An unfamiliar slot is not a discovery by default — the light user’s re-ranker list is unfamiliar and wrong. **How methods differ.** PPR at α = 0.3 keeps most mass at the restart, so it is the personal count plus co-viewed neighbours; more diffusion pulls it to popularity (α = 0.85: 0.29821). ALS loses frequency order (above). Content helps where co-view is thin only if the user’s few programmes have rich metadata; for the light user it finds talk shows like the watched one and misses. Association rules were not implemented (§2), so I claim nothing about what they would catch. Popularity is already strong for news-heavy viewers (heavy user 0.52) because five daily news shows top every list. **Choice.** The re-ranker, as the main “your shows this week” rail: its gain over PPR is [+0.03347, +0.04036] MAP@5 (95% interval, round 1), 3,810 of 4,254 validation users with ≥ 1 hit (popularity 3,214, PPR 3,700), and 117 distinct programmes across the 6,300 slots of the final test file (popularity 5). Its cost is one feature build and one boosted model per refresh (149 s for 4,254 users on 4 CPUs); PPR (28.6 s) is the fallback when that budget is not available, and users with almost no history should get PPR or popularity, labelled as such. **Limitations supported by the results.** Only 10.8% of schedule rows are labelled, so most airtime — films included — is invisible as items. 2,679 of 4,254 validation users have a tie at place five, so part of every MAP@5 is tie-

breaking noise. The log is this platform's only: absence is not "never watched". The validation–public gap shows the test cohort differs from mine. No online effect, A/B result or revenue is claimed; every number above is offline.

6. AI Usage Disclosure

Tools and session. Claude Code 2.1.308 (CLI), model Claude Opus 5.5 (`claude-opus-5-5`), one session `35d97a0d-5715-4a99-b207-aca036c0609f`, submitted unmodified as `eugene_shmat_exam_session.json`; when this PDF was compiled it held 12 prompt entries of mine (my `!` commands included) plus 5 messages typed while the agent worked, 400 assistant messages and 160 tool calls (128 Bash, 20 Write, 8 Edit, 4 Read; `eval/session_stats.py`); the session continues after that only to commit, push and copy the log. The agent wrote every file in `src/`, `eval/`, `kaggle/` and `report/` of the repository, ran the private Kaggle kernel `shmate/sweettv-exam-val` (versions 1–3) and made the eight submissions under my blanket approval of up to eight; it modified no data file. It was used for code generation, the data audit, debugging and the first draft of this report, which I read. **My direction, visible in the session.** I gave the process rules, approved the sources and the submissions, found the IR-book URL I had been offered broken (the assistant's `curl` then returned 000 and the source was dropped), sent the list of ideas that became the re-ranker's feature groups and the formula scorer, and expected a personal ranker to land near 0.4 public when the assistant had only tested recency decay — the personal list then returned 0.40431. I also moved the heavy runs to Kaggle kernels, and I opened the version-3 kernel log on Kaggle myself and pasted it into the session: its values (0.38012, 0.38095, interval [−0.00111, +0.00270], the sha256 of `f_rerank_v2.csv`) equal the output the assistant downloaded. **What I personally verified — the hand chain on user 70601421848869461 (`eval/hand.py`, inputs only, my answers in the session before the code's).** (a) Join: the view 16:35:30–17:15:50 covers the airing 16:45:00 + 1500 s, overlap 1500 s, screen time 1.0, watched. (b) Personal score of programme 2400480: 0.7744 + 0.7879 + 0.9715 = 2.5338, times $1434^{0.25}$ = 6.1537, gives 15.5925; the code prints 2.5340 and 15.5933 — the gap is my rounding of $0.5^{5.5/16}$ = 0.78799 down to 0.7879. (c) AP@5: hits at positions 1 and 2, (1 + 1) / 5 = 0.4000, equal to the code. (d) Reading: I skimmed [1], [2], [3], [4] and know [5] from HW6; a fourth source I had approved (Jeh and Widom, WWW 2003) was dropped because its only free URL (Stanford InfoLab) stopped answering during the session. The assistant checked authors, venue and pages against Crossref ([4]) and the JMLR page tags ([6]), and read in the PDF of [4] the confidence $1 + \alpha r$, the log variant and $\alpha = 40$. (e) I re-ran `eval/verify.py` myself from my prompt (stdlib only, raw CSVs): six toy cases pass, the validation target matches the pipeline for all 4,254 users, and the MAP@5 of all eight lists, the final 0.38095 included, is reproduced. **Checked by the assistant's scripts only:** the second join, the cross-environment sha256 match and `check_numbers.py`. **Errors of the assistant caught in the session.** It launched four grids at once on my 8 GB laptop, so one ALS configuration took 262.9 s and the first full re-ranker run died with no output; its parallel Kaggle launch raced on the cache (failure case 2); its first `curl` without `-L` saved an HTML redirect as the ALS paper; and its progress note ("3 minutes" elapsed, against an estimate of 30–40) read like a promise, so I asked whether the kernel had hung; it had not (its four steps took 48, 217, 885 and 341 s). Its first push carried an e-mail address that GitHub attributes to another account; I noticed, and the commit was re-authored to my account and force-pushed. **What I trusted without verification.** The bootstrap intervals (1,000 resamples), the re-ranker ablation values, and the PPR, ALS and content scores fed to the re-ranker; I did not read the scripts or the kernel code line by line.

References

[1] ogurbych (Kaggle host), "sweet.tv --- TV program recommender (Kaggle community competition, host ogurbych; Overview, Data and Evaluation pages)." Accessed: Oct. 22, 2026. [Online]. Available: https://www.kaggle.com/competitions/sweettv-tv-program-recommender

[2] B. Hamner, *Metrics: machine learning evaluation metrics (Python ml_metrics, average_precision.py: apk, mapk)*. Accessed: Oct. 22, 2026. [Online]. Available: https://github.com/benhamner/Metrics

[3] S. Jin, "Final Exam Quiz (AVITO): TV Program Recommendation on sweet.tv (LLM4Rec course, HSE University)." Oct. 2026.

[4] Y. Hu, Y. Koren, and C. Volinsky, "Collaborative Filtering for Implicit Feedback Datasets," in *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM 2008)*, Dec. 2008, pp. 263–272. Accessed: Oct. 22, 2026. [Online]. Available: https://yifanhu.net/PUB/cf.pdf

[5] S. Jin, "LLM4RecSys 06. Graph-based RecSys (lecture slides: PageRank and Personalized PageRank; LLM4Rec course, HSE University)." 2026.

[6] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011, Accessed: Oct. 22, 2026. [Online]. Available: https://jmlr.org/papers/v12/pedregosa11a.html